

[Marcar como hecha](#)

Más con AngularJS

En el artículo anterior hemos visto, en el último código, ya la creación de un `angular.module` y utilizar un primer `angular.controller`. Son conceptos que incorporarlos, no ayudará mucho a la hora de trabajar con las versiones actuales de angular, en las que estos controladores y el modulo se crean de modo automático.

El este código, no nos contentamos con crear un **controlador**, sino que además incluimos en una función de él unas cuantas funciones (añadir tareas pendientes, verlas, archivar tareas realizadas..)

Vamos a ir un paso atrás. Comenzaremos, a modo de repaso, por crear una iteración con `ng-repeat`, a partir de un array *pensamientos* declarado en el momento en que declaramos la aplicación de angular, en la etiqueta `html` .

Una vez declarado, este array se puede rellenar interactivamente en un formulario, e ir mostrando el contenido en la medida que se va llenando.

Vamos a verlo:

```
<!DOCTYPE html>
<html lang="en" ng-app ng-init="pensamientos = [];">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>
<body>
<div >
<h1>Altavoz AngularJS</h1>
<p>
¿Qué hay de nuevo?
<br />
<input type="text" ng-model="nuevoPensamiento" />
<input type="button" value="Agregar" ng-click="pensamientos.push(nuevoPensamiento);
nuevoPensamiento = '';" />
</p>
<h2>Pensamientos que has tenido</h2>
<p ng-repeat="pensamiento in pensamientos" ng-init="paso = $index">
Pensaste esto: {{pensamiento}} (Iteración con índice {{paso}})
</p>
</div>
</body>
</html>
```

Bien, funciona, pero ahora que ya habéis visto las posibilidades de un controlador, seguro que os interesa aprender más de esto. Como decíamos iremos despacio. Comenzaremos por crear un controlador que nos permita solamente declarar una serie de variables que tendrán ámbito (scope) de nuestra aplicación.

AngularJS, fue evolucionando hasta que el atributo de `$scope`, resultaba menos conflictiva si estaba "minimificado". Esto es, que el controlador declaraba un array que comenzaba con `$scope`, y luego declaraba dentro lo necesario.

```
.module('pruebaApp', [])
```

```
.controller('pruebaAppCtrl', ['$scope', function($scope){
```

```
$scope.algo = "Hola Angular, usando controller!";
```

```
}}]);
```

?

En este caso, en la llamada al método que crea el controlador, `controller()`, es un array que nos sirve para inyectar las

dependencias.

```
['$scope', function($scope){ ... }]
```

En este caso le estamos diciendo que el primer parámetro del controlador es '\$scope', gracias al primer elemento del array. Como las cadenas como '\$scope' no se alteran después de la minimización, podemos estar seguros que le llegarán perfectamente a Angular. Entonces sabrá que en el primer parámetro de la función constructora del controlador esperas recibir el '\$scope'. Esto será así independientemente del nombre de la variable que se uses como primer parámetro. Es cierto que la sintaxis parece un poco rebuscada, pero poco a poco te acostumbrarás, pues realmente es muy repetitivo.

Si el código anterior lo guardamos en un documento llamado ang.js, y lo llamamos en el siguiente html, veremos que el controlador puede ser invocado como atributo de un div (veremos que en las versiones actuales esto ha evolucionado hasta convertirse en una etiqueta de html). Lo que nos interesa, es posible incorporar el controlador allí donde lo queramos.

Por otro lado, en este html, veremos que la llamada a la librería de angularJS se hace al pie del body, del modo actual como se llama a cualquier comportamiento de js: veamos el código:

```
<!DOCTYPE html>
<!-- nombre del modulo que quieres ejecutar -->
<html lang="en" ng-app="nuevaApp">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
<div ng-controller="pruebaAppCtrl">
  {{algo}}
</div>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.2.24/angular.min.js"></script>
<script src="ang.js"></script>
</body>
</html>
```

Un controlador más complejo

Ahora que hemos cogido el gusto a los controladores, nos embarcaremos en un ejemplo un poco más complejo, crearemos una pequeña aplicación que vaya acumulando valores sumando y restando, desde dos funciones dentro del controlador, a partir de un mismo formulario y sin tener que recargar la página ni usar complicados métodos get o post. **Empecemos por ver el controlador**, que debemos guardarlo en un documento ang2.js

```
angular
.module('acumuladorApp', [])
.controller("acumuladorAppCtrl", controladorPrincipal);
function controladorPrincipal(){
  //esta función es mi controlador
  var scope = this;
  scope.total = 0;
  scope.cuanto = 0;
  scope.sumar = function(){
    scope.total += parseInt(scope.cuanto);
  }
  scope.restar = function(){
    scope.total -= parseInt(scope.cuanto);
  }
};
```

- En la segunda línea creamos el módulo, indicando el nombre (que es igual a lo que pondremos como valor en la directiva ngApp del HTML).
- En la tercera línea, con un encadenamiento (chaining) definimos el **controlador**. Indicamos como primer parámetro el nombre del controlador, que definiremos en la directiva ngController del HTML, y **como segundo parámetro colocamos la función que se encargará de construir el controlador**. Indicamos el nombre de la función simplemente, **sin los paréntesis, pues no es una llamada a la función sino simplemente su nombre**.
- Luego se define la función del controlador. Esa función es capaz de escribir datos en el scope, así como métodos.

?

- En la función accedemos al scope por medio de **"this"**. Fíjate que en la primera línea de la función tienes `var scope = this`; esto es simplemente opcional y se puede hacer para mantener la terminología de AngularJS de llamar scope a lo que vas generando en el controlador, pero podría perfectamente referirme a él todo el tiempo con `"this"`.
- En el scope inicializo dos valores, total y cuanto, mediante `scope.total=0` y `scope.cuanto=0`.
- Luego genero dos métodos que usaremos para los manejadores de eventos de los botones de sumar y restar. Esas funciones tienen el scope disponible también y en su código se accede y modifica a los datos del scope.

Veamos el HTML:

```
<!DOCTYPE html>
<!-- nombre del modulo que quieres ejecutar -->
<html lang="en" ng-app="acumuladorApp">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
<div ng-controller="acumuladorAppCtrl as vm">
<h1>Acumulador</h1>
<h2>Control de operación:</h2>
¿Cuánto? <input type="text" ng-model="vm.cuanto" size="4" />
<br />
<input type="button" value="+" ng-click="vm.sumar()"/>
<input type="button" value="-" ng-click="vm.restar()"/>
<h2>Totales:</h2>
En el acumulador llevamos <span>{{vm.total}}</span>
</div>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.2.24/angular.min.js"></script>
<script src="ang2.js"></script>
</body>
</html>
```